

A Causal Audit of Baselines for LLVM Code-Size Optimization

Dimitrije Pešić

School of Electrical Engineering, University of Belgrade

Belgrade, Serbia

pd230014d@student.etf.bg.ac.rs

Abstract—A large improvement over a compiler baseline can depend on how the baseline’s input is prepared. We investigate this effect in an LLVM phase-ordering experiment using the original CompilerGym service. Adding function size attributes to both reference and candidate inputs changes a fixed portfolio’s instruction-count gain on 120 NPB modules from 23.84% to 0.08%. A controlled intervention that prevents loop unrolling removes 12,560 of the 12,639 additional instructions in the original reference. Measurements on fixed optimized IR separately identify an attribute-dependent code-generation effect. The corrected comparison still retains 5.76% savings in code-section bytes, so the instruction-count result does not determine the native-size result. Paired measurements on three suites, a small source-build control, and a learned-versus-random selection check delimit the finding. We provide per-module records and an executable example to make baseline preparation, causal interventions, and size definitions inspectable. The study concerns the audited protocol and does not establish errors in other CompilerGym-based results.

Index Terms—compiler optimization, LLVM, code size, phase ordering, reproducibility

I. INTRODUCTION

Choosing a sequence of compiler optimization passes to reduce code size is a long-studied problem [1]. Such sequences are often evaluated by comparing the resulting intermediate representation (IR) with a standard optimization pipeline. CompilerGym [2] makes this experiment accessible through a common interface, including an LLVM `-Oz` reference and a fast instruction-count objective. Learned methods can select passes or choose from a portfolio of sequences [3]. The practical motivation is smaller deployed code, but two distinctions matter: applying a named pipeline to existing IR is not necessarily equivalent to compiling source code with that optimization level, and counting IR instructions is not the same as measuring the generated machine code.

We encountered both distinctions while evaluating a fixed portfolio and a learned pass-ordering policy. On 120 NPB modules, eight portfolio candidates appeared to reduce the summed instruction count by 23.84% relative to CompilerGym’s `-Oz` reference. Giving both sides the function attributes used for size optimization reduces that advantage to 0.08%. Such a change warrants an explanation before interpreting the original result as an improvement over compilation for size.

This paper asks: *which part of the apparent improvement is sensitive to input preparation, and which compiler decisions explain that sensitivity?* We keep the original LLVM service

and candidate sequences fixed. A paired experiment measures the attribute effect; an intervention on loop metadata tests unrolling; and attributes added only after IR optimization isolate their effect during code generation. A real function makes the mechanism visible, while measurements on other suites and source builds limit its scope.

Our contribution is a controlled, reproducible case study of an evaluation result. Size attributes and their effects are established compiler behavior. We quantify their impact on the conclusion of one experiment and distinguish an IR-level effect from a native-code effect. The original service reference remains a well-defined benchmark; our concern is what a gain against it supports about source-level size optimization.

II. BACKGROUND AND RELATED WORK

LLVM function attributes carry optimization intent alongside the IR. The `optsize` and `minsize` attributes request size-oriented decisions [4]; selecting a pipeline alone does not add them to an arbitrary input module. In LLVM 10, the unroller explicitly checks function size attributes when choosing its expansion thresholds [5]. The pipeline and function-level settings must therefore be distinguished.

This is not an undocumented concern. CompilerGym’s benchmark-construction code discusses how the frontend optimization level changes attributes, including `minsize` and `optsize` [6]. An earlier dataset issue documented `noinline` attributes left by `-O0` preparation [7]. That issue concerns a different attribute and transformation, but illustrates why benchmark preparation is part of the experimental protocol. More generally, innocuous-looking details of an experimental setup can bias conclusions about compiler optimizations [8].

CompilerGym also exposes object-size measurements and documents the limitations of instruction count as a proxy [9], and learned size optimization integrated into LLVM is trained on native size reduction [10]. The coreset approach [3] demonstrates learned selection over pass-sequence candidates; neither portfolios nor byte-based selection originate in this study. Our question is the magnitude and mechanism of a baseline-sensitive result, with a secondary check of whether an IR advantage survives the choice of a native-size objective.

TABLE I
INTERVENTIONS. ALL IR OPTIMIZATION USES THE ORIGINAL SERVICE.

Case	Change relative to the canonical module
P	None; the original experiment’s preparation.
U	Prevent unrolling through input loop metadata.
A	Add size attributes to function definitions.
AU	Combine A with the unrolling intervention.
U ⁺	Add size attributes to U’s optimized output; run only code generation afterward.

III. EXPERIMENTAL METHOD

A. Service and paired comparison

We use the unmodified CompilerGym 0.2.5 service and its bundled LLVM 10 on x86-64. We invoke the service’s built-in baseline operation and require the exported module’s static IR instruction count (IC) to equal its `-Oz` observation. The standalone `opt -Oz` command constructs a different pipeline and is not substituted for this reference. Exact API calls and toolchain hashes are recorded in the artifact.

The paired audit covers 210 modules: 120 NPB [11], 40 MiBench [12], and 50 BLAS [13]. Membership follows the existing experiment’s per-suite lists and its unoptimized-IC cap of 6,000. The first eight sequences of the previously mined portfolio are frozen before the audit. Each sequence has 45 actions in a 36-pass space; it is executed through service actions. A random control uses eight uniformly drawn sequences per module, with the same draws in both preparations. The candidate whose final IR has the smallest IC is selected, with the earliest candidate breaking ties. There is no reference fallback in this comparison. All modules and all candidates complete in both preparations.

The original preparation, P, is the canonical module exported by the environment, without size attributes. Preparation A adds `minsize` `optsize` to function definitions only. Declarations, call-site attributes, target triple, data layout, and the other attributes are preserved. Both the reference and every candidate start from the same prepared module in each condition. Inputs are loaded through content-specific file URIs, and their full disassembled IR is checked after reloading, avoiding stale benchmark-cache entries.

B. Interventions and fidelity checks

The causal audit uses the same 120 NPB modules and the preparations in Table I. U adds `llvm.loop.unroll.disable` metadata to every natural input loop and then runs the original service reference. The pass pipeline remains unchanged. A fourth condition, AU, applies the same intervention to A. It checks whether unrolling still contributes under the size attributes.

For optimization remarks, we construct a small replica of the service’s pass-manager setup, linked against the same LLVM release. Its complete output IR and measured costs must reproduce the service for each preparation before interpreting its remarks. A builder-level unrolling switch and a zero-threshold setting provide additional diagnostics; the input-

TABLE II
SAVINGS OVER THE SERVICE REFERENCE (%). CANDIDATES ARE SELECTED BY IC IN EVERY COLUMN. P/A: WITHOUT/WITH SIZE ATTRIBUTES ON BOTH SIDES.

Suite (<i>n</i>)	Method	IC		Code bytes	
		P	A	P	A
NPB (120)	Portfolio	23.84	0.08	23.71	5.76
NPB (120)	Random	22.35	-1.87	24.53	6.46
MiBench (40)	Portfolio	1.94	-0.70	3.32	1.47
MiBench (40)	Random	0.62	-2.06	2.39	1.00
BLAS (50)	Portfolio	2.89	2.89	0.87	0.95
BLAS (50)	Random	2.26	2.26	0.80	1.14

metadata intervention is primary because the builder switch also changes cleanup passes. These diagnostics and their disagreements are retained in the artifact.

The code-generation control U⁺ changes only the two size attributes on U’s already optimized IR before running `llc`. An analogous late-attribute control on P and an attribute-removal control on A test the dependence on the optimized IR. The complete IR, after normalizing these attributes, must remain unchanged in each such control.

C. Measurements and interpretation

Each measurement includes IC, code-section bytes (summed `.text` sections), and Berkeley text (the `text` column of `llvm-size`). The latter also includes other read-only allocated sections, such as constants and unwind data. Objects are generated by the bundled `llc` with the module’s existing target settings. Both byte metrics describe relocatable objects, not a linked executable or its runtime memory footprint.

For a cohort S , the reported saving is

$$G(S) = 100 \left(1 - \frac{\sum_{i \in S} C_i}{\sum_{i \in S} B_i} \right), \quad (1)$$

where B_i and C_i are reference and selected-candidate costs under the same preparation. Positive values mean smaller output. This ratio weights large modules more; it is not the average per-module saving. We report module outcomes as well as totals and do not treat related modules as independent projects.

IV. RESULTS

A. Baseline sensitivity is large on NPB

Table II reports both preparations and both methods. The portfolio’s NPB IC gain changes from 23.84% to 0.08%, while code-section savings change from 23.71% to 5.76%. The random control has the same qualitative behavior: its apparent IC gain becomes a loss, while a code-section saving remains. The effect is smaller on MiBench and absent in the BLAS IC comparison. Thus neither the magnitude of the NPB effect nor the disappearance of the IR gain should be generalized to all suites or all size metrics.

For attributed NPB inputs, the portfolio wins/ties/loses on 34/37/49 modules in IC, and 79/13/28 in code bytes. The near-zero summed IC gain therefore does not imply identical results

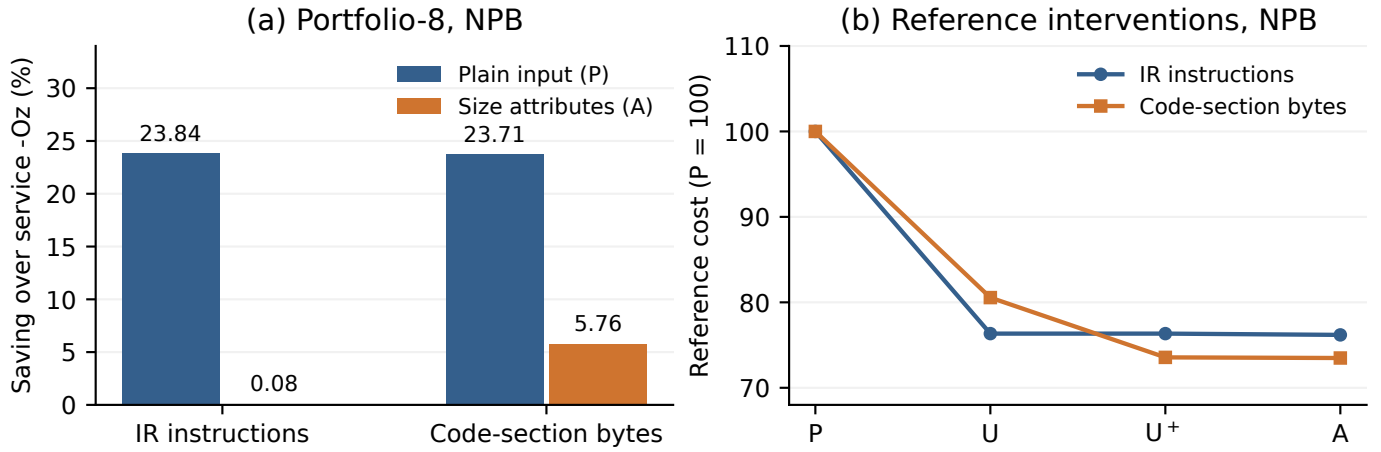


Fig. 1. The main effect and its mechanism. (a) Portfolio-8 savings on the 120 NPB modules when selected by IC, before and after size attributes are added to both sides. (b) Reference costs under the interventions, normalized separately to P. U⁺ changes attributes only before code generation. These are deterministic cohort totals, not estimates of a population mean.

per module. Candidate outputs can also change when attributes are added; the paired result is not a change of the denominator alone.

B. Unrolling explains almost all reference IC inflation

The original reference contains 53,085 instructions; U has 40,525, and A has 40,446. Preventing unrolling removes 12,560 of the 12,639 instructions separating P and A. The replica reports 449 fully unrolled loops on 43 modules under P, and none under A. On each of these 43 modules, U and A have the same IC and the same complete IR after normalizing size attributes and loop metadata. AU and A have identical metrics throughout the cohort. The residual 79 instructions occur on 5 modules with no unrolling; IR differences suggest attribute-sensitive inlining and instruction combining, which were not separately ablated.

This evidence goes beyond attributing every difference to an optimization flag. A targeted intervention reproduces almost the entire reference IC change in the original service. It measures the effect of preventing unrolling and the subsequent pipeline behavior; it is not a universal additive partition of compiler decisions.

C. Code generation accounts for a further byte effect

For code sections, P, U, and A total 291,616, 234,913, and 214,296 bytes. U⁺ has exactly U’s IC but is 20,402 bytes smaller, leaving it only 215 bytes above A. For 114 of the 120 modules, U and A have identical normalized IR and U⁺ produces the same object bytes as A. Removing the attributes from A before code generation reverses this equality on those modules.

The attribution depends on the order of interventions. Adding the attributes late to P saves 14,902 bytes; adding them late to U saves 20,402. We therefore report these controlled differences rather than assigning an invariant percentage of the total byte gap to each cause. Berkeley text likewise falls from 348,878 through 290,559 to 267,814 bytes, but it is a different quantity from executable code sections.

D. A concrete function exposes the mechanism

We extract `transfb_nc0` unchanged from NPB module 116 using `llvm-extract`. Its original C source is unavailable; no source is reconstructed. The function contains a 5×3 loop nest. Under P the nest expands to 15 copies of its body, leaving 90 instructions and 377 function bytes. U retains the loops at 25 instructions and 119 bytes. A has the same IC but only 87 bytes; adding attributes late to U produces that same size. The function body and size match those in the full-module objects, so extraction did not create this effect.

The artifact includes the input IR, transformed IR, objects, and a driver. All variants agree bit for bit on four fixed pseudorandom input vectors and all 25 outputs per vector. This is a regression check, not a semantic-equivalence proof. The example illustrates the cohort-level intervention; it is not a separate independent benchmark.

E. Source builds and candidate selection bound the claim

Adding attributes does not reproduce every decision of a source build. For 12 CHStone programs [14], the plain service, attributed service, and direct `clang -Oz` builds contain 38,315, 32,410, and 32,123 code-section bytes, respectively. The attributed service is 0.89% larger than the direct build in aggregate, but GSM remains 12.81% larger. Closeness of the pooled sum is not evidence of per-program equivalence. This source control is confined to CHStone.

A secondary audit examines 347 programs under A, with eight candidates per generator: three stored GNN checkpoints and five random realizations. Selecting the same pools directly by code bytes yields GNN savings of 2.84–3.66% and random savings of 2.82–3.68%. The GNN has a lower summed IC than every random realization under IC selection, but no consistent code-byte advantage across generator pairings under byte selection. This is not evidence of equivalence, and the policy was not retrained under the new preparation. The broader Berkeley-text metric gives different conclusions: Berkeley-first GNN selection saves 1.35–2.06% even without NPB, whereas

code-first code-section savings are close to zero or negative. Full paired records are provided separately to keep this check distinct from the causal NPB experiment.

V. SCOPE AND PRACTICAL IMPLICATIONS

The strongest result is specific to LLVM 10, the original service pipeline, the frozen candidates, and the stated cohorts. The NPB modules are related and the cohort was inherited from the earlier experiment. We do not estimate how often this effect occurs across arbitrary software, and we have not audited the references of other publications. The direct source-build control does not establish equivalence for the NPB inputs. No end-to-end optimization time, execution-time improvement, or linked-binary saving is claimed.

IC agreement is checked exactly, but bitcode serialization can differ for identical IR. One unselected candidate in the secondary audit also has two object hashes with equal measured sizes. We retain these distinctions rather than equating cost reproduction with universal bitwise determinism. Fixed-vector tests do not prove correctness for all inputs. The candidate-selection ranges describe the observed checkpoints and realizations, not confidence intervals or the outcome of retraining.

For a size-oriented evaluation, the input attributes should be stated alongside the pass pipeline. Reference and candidate inputs should have matched preparation, with source-level comparisons where possible. IR counts, code sections, and broader object size should be reported as separate objectives. A random control at the same candidate budget helps distinguish the effect of learned generation from the effect of search and selection. These practices define the comparison; they do not require every size-oriented task to use one universal attribute set.

VI. CONCLUSION

In the audited experiment, input preparation changes the apparent portfolio IC advantage on NPB from 23.84% to 0.08%. A targeted unrolling intervention explains nearly all of the reference’s IC change, while a fixed-IR control reveals a further code-generation effect. A 5.76% code-section saving nevertheless survives. The result demonstrates why a gain over an IR pipeline must be interpreted with its input attributes and its actual size metric, rather than automatically as a gain over source-level compilation for size.

Artifact. Source, per-module records, numerical checks, and the executable example are available at <https://github.com/dimitrijepesic/compiler-opt>. The figure and numerical macros are generated from recorded results; the repository separates record verification from rerunning the compiler.

ACKNOWLEDGMENT

The author thanks Prof. Zaharije Radivojević for early discussions of this work.

REFERENCES

- [1] K. D. Cooper, P. J. Schielke, and D. Subramanian, “Optimizing for reduced code space using genetic algorithms,” in *Proc. ACM SIGPLAN LCTES*, 1999, pp. 1–9.
- [2] C. Cummins *et al.*, “CompilerGym: Robust, performant compiler optimization environments for AI research,” in *Proc. IEEE/ACM CGO*, 2022, pp. 92–105.
- [3] Y. Liang *et al.*, “Learning compiler pass orders using coresets and normalized value prediction,” in *Proc. ICML*, ser. Proc. Machine Learning Research, vol. 202, 2023, pp. 20 746–20 762.
- [4] LLVM Project, “LLVM 10 language reference: Function attributes,” <https://releases.llvm.org/10.0.0/docs/LangRef.html>, 2020.
- [5] —, “LoopUnrollPass.cpp, release llvmmorg-10.0.0,” <https://github.com/llvm/llvm-project/blob/llvmmorg-10.0.0/llvm/lib/Transforms/Scalar/LoopUnrollPass.cpp>, 2020.
- [6] CompilerGym Contributors, “Benchmark construction, version 0.2.5,” https://compiler-gym.com/_modules/compiler_gym/envs/llvm/llvm_benchmark.html, 2022.
- [7] C. Cummins, “cBench benchmarks have ‘noinline’ attribute set, issue 110,” <https://github.com/facebookresearch/CompilerGym/issues/110>, 2021.
- [8] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!” in *Proc. ACM ASPLOS*, 2009, pp. 265–276.
- [9] CompilerGym Contributors, “LLVM environment reference, version 0.2.5,” <https://compiler-gym.com/llvm/>, 2022.
- [10] M. Trofin, Y. Qian, E. Brevdo, Z. Lin, K. Choromanski, and D. Li, “MLGO: A machine learning guided compiler optimizations framework,” arXiv:2101.04808, 2021.
- [11] D. Bailey, T. Harris, W. Saphir, R. van der Wijngaart, A. Woo, and M. Yarrow, “The NAS parallel benchmarks 2.0,” NASA Ames Research Center, Tech. Rep. NAS-95-020, 1995.
- [12] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, “MiBench: A free, commercially representative embedded benchmark suite,” in *Proc. IEEE WWC-4*, 2001, pp. 3–14.
- [13] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh, “Basic linear algebra subprograms for Fortran usage,” *ACM Trans. Math. Softw.*, vol. 5, no. 3, pp. 308–323, 1979.
- [14] Y. Hara, H. Tomiyama, S. Honda, H. Takada, and K. Ishii, “CHStone: A benchmark program suite for practical C-based high-level synthesis,” in *Proc. IEEE ISCAS*, 2008, pp. 1192–1195.